

AI DevFlow Studio

文字主体 × 简单漫画风格 · 9 页研发复盘

把开发过程中遇到的问题、解决思路、落地动作和结果讲清楚。

讲法调整

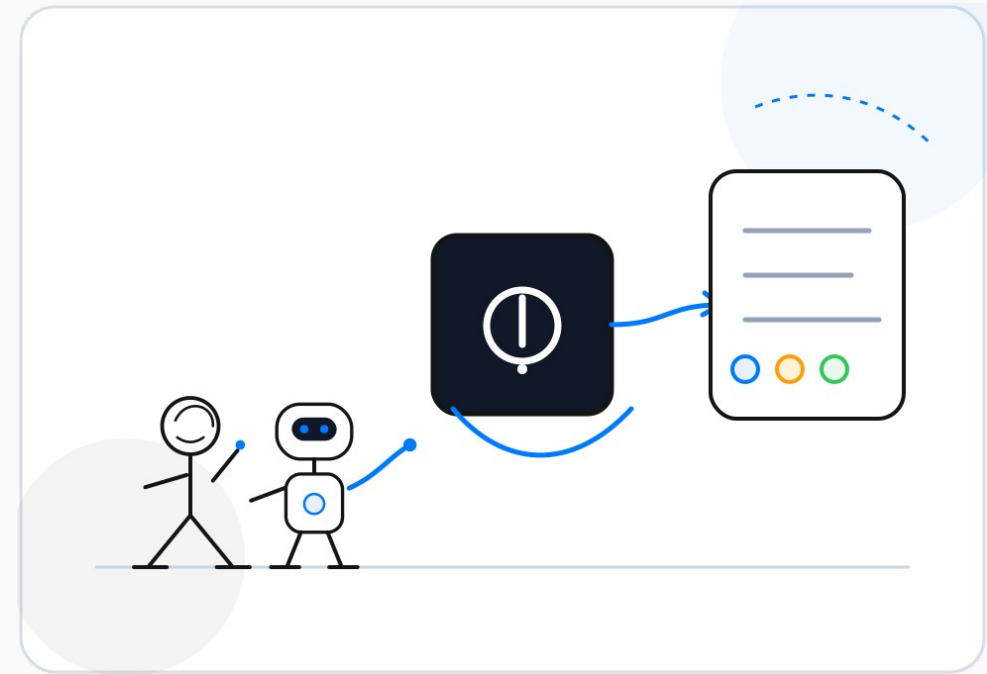
不再把它画成故事；每页都围绕“问题 → 思路 → 落地 → 结果”展开。

视觉原则

漫画只作为示意和记忆点；主体内容保留为可编辑文字、流程和判断。

核心命题

让 AI 研发从黑盒调用，走向证据驱动的团队交付。



Knowledge

Agent

Gate

Evidence

定位：探索型 Demo ，不宣称商业化完成；只验证一条可行路径。

基于仓库情况，先搭团队知识底座

多仓库不是简单堆文件；必须先让前端、后端、数据库知识有统一入口。

遇到的问题

前端、后端、数据库分散在不同代码仓库，Agent 直接读仓库容易丢上下文，也很难复用。

解决思路

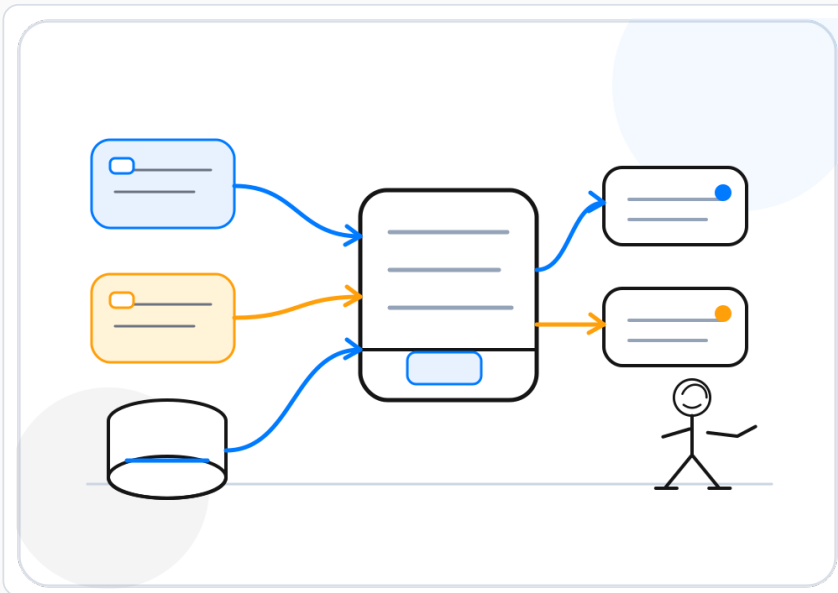
用一个知识仓库连接多个代码仓库，把可复用理解沉淀成团队知识底座。

落地动作

按知识域组织：System Knowledge 记录技术边界；Business Knowledge 记录业务术语、规则和流程。

阶段结果

多仓协作有了统一上下文入口，需求、设计、Review、Gate 都能引用同一套材料。



漫画示意：多个代码仓库接入同一个知识入口。

本页结论

先解决“上下文从哪里来”，后续 Agent 才有共同依据。

多仓

知识域

统一入口

ISCQ / WeQ 提炼， isctPilot 引导

提炼工具能提高效率，但不能把“提炼结果”直接当成可信事实。

遇到的问题

工具能从仓库抽取大量材料，但结果可能只是片段、摘要或假设；没有确认就不能进入团队决策。

解决思路

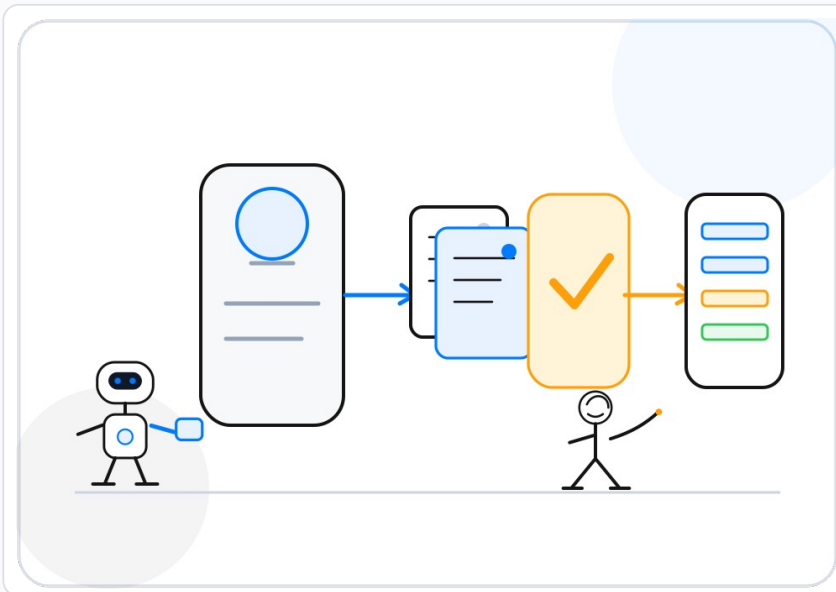
引入 Candidate → Review / Gate → Confirmed 的状态流，让知识先被看见，再被确认。

落地动作

isctPilot 引导建库；ISCQ / WeQ 生成业务领域和技术领域材料；再沉淀术语与关系。

阶段结果

Confirmed Knowledge 才能作为需求澄清、方案设计、Agent 上下文和 Gate 判断的证据。



漫画示意：工具提炼材料，Review / Gate 才能盖章确认。

本页结论

知识从“候选材料”走到“可引用证据”，中间必须有确认。

候选知识

Review

确认知识

从 US 拆出可执行的知识材料

原始 User Story 只是入口，真正可执行的是术语、约束、影响面和测试关注点。

遇到的问题

US 直接交给 Skill，容易遗漏边界、业务词义和系统影响，测试用例也缺少依据。

解决思路

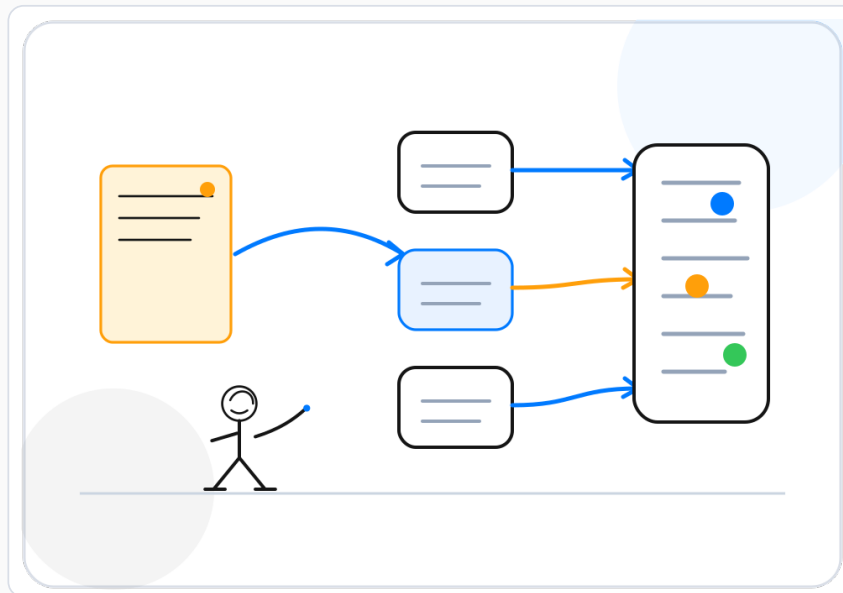
先把 US 拆成需求目标、业务术语、技术影响、数据约束和待确认问题。

落地动作

拆解结果写入对应知识域：专用术语、术语关系、规则、接口、数据模型、测试线索。

阶段结果

需求方案、开发实现、测试用例都有可引用材料，而不是依赖一次性提示词。



漫画示意：一张 US 被拆成术语、关系、规则和测试线索。

本页结论

需求不再是一句提示词，而是一组可复用、可追溯材料。

[US 拆解](#)[术语关系](#)[测试线索](#)

CRI Skill 的问题：不是不会做，而是看不见

Skill 调用越强，越需要可观测、可追溯、可参与。

问题 1 • 缺乏可观测性

无法对 Skill 的调用流程进行追踪，Skill 的路由也变得不再可控。

问题 2 • 缺乏追溯手段

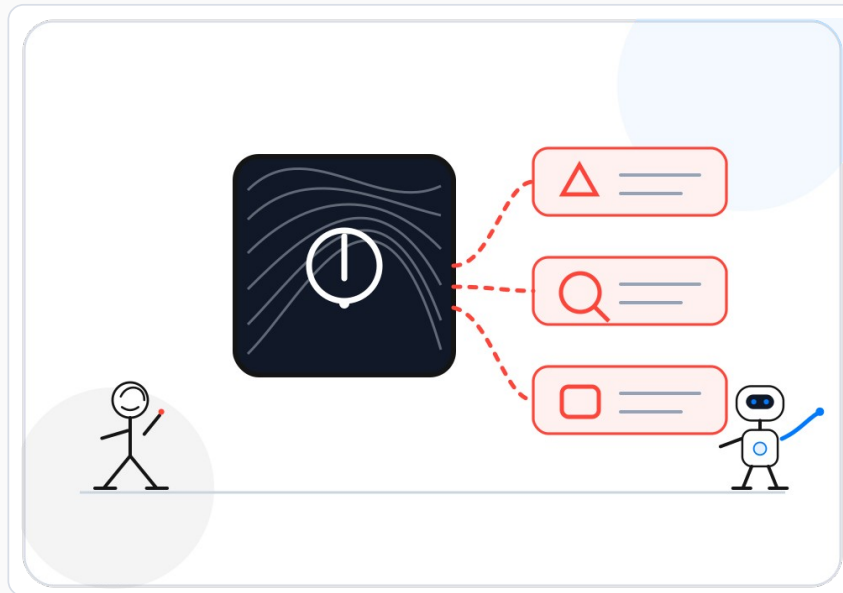
尽管有 pilot 工具，但它无法公测；出现问题后缺少可复盘的依据链。

问题 3 • 操作体验不佳

需求、测试和开发都存在类似问题；只能在 CI 中操作，对 BA / QA 并不友好。

解决方向

把 Skill 能力放进可视化流程：Trace 记录调用，Evidence 保存证据，Gate 让人参与判断。



漫画示意：黑盒越大，团队越难判断它为什么这么做。

本页结论

核心卡点不是能力缺失，而是缺少可治理的执行入口。

不可观测

难追溯

CI-only

换思路：先设计流程产品，再生成工程界面

如果只优化提示词，问题仍在黑盒里；先确定信息架构和交互，流程才可用。

遇到的问题

单纯依赖 Skill / CI，无法形成 BA、QA、开发都能理解的共同界面。

解决思路

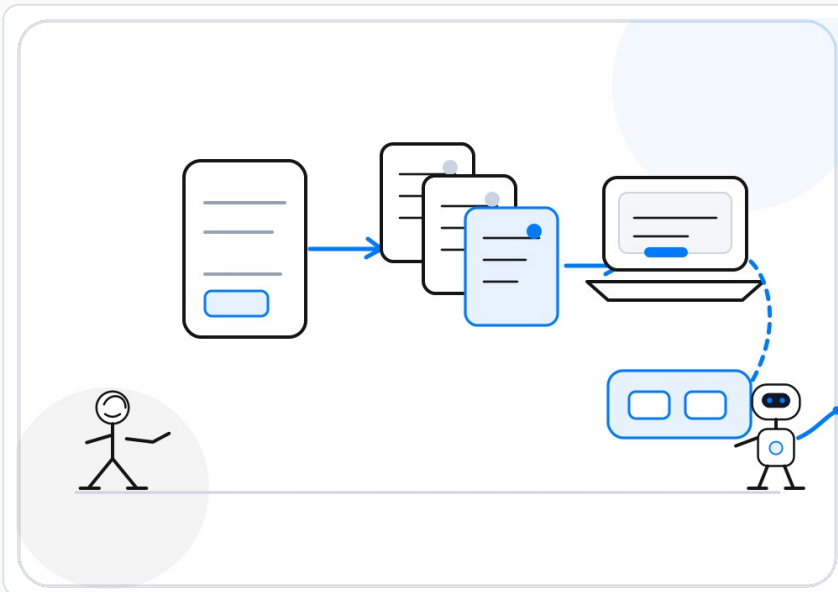
用 Open Design 先做 HTML 原型，验证 Run、Node、Gate、Evidence、Trace 的信息结构。

落地动作

提示词迭代 3 个版本，再用 Codex 细化提示词与页面结构；HTML 产物再转换成 React。

阶段结果

设计不只是图，而是工程蓝图：每个界面都对应流程节点、证据和协作动作。



漫画示意：从流程草图到 Prompt 迭代，再到 HTML / React。

本页结论

设计不是装饰，而是把治理思路翻译成可操作界面。

Open Design

3 版 Prompt

HTML → React

Electron 承接 Mac / Windows 的工作台能力

研发流程要真正落地，不能只停在浏览器原型或 CI 日志里。

遇到的问题

本地仓库、测试命令、文件路径和凭证都在开发者机器上；Mac / Windows 行为还不一致。

解决思路

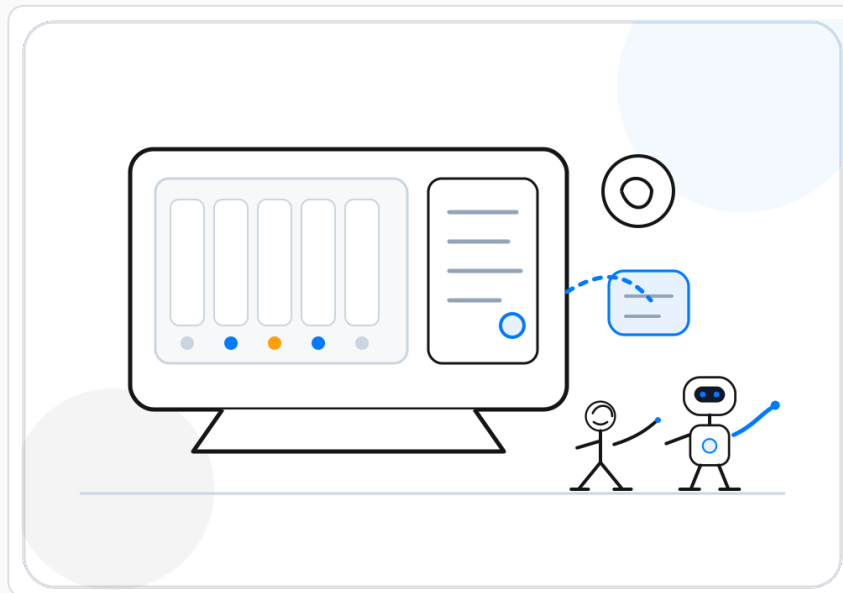
用 Electron 做跨端桌面入口，React 负责交互，Node / Electron 承接本地执行和安全边界。

落地动作

本地状态用 SQLite；流程看板展示 Run / Gate / Evidence；Inspector 展示证据、Trace 和风险影响。

阶段结果

BA / QA / 负责人不必读 CI 黑屏，也能看到需求、设计、测试和交付证据。



漫画示意：桌面工作台把流程看板和证据 Inspector 放到一处。

本页结论

把 CI 黑屏里的结果，变成团队能看懂、能参与的工作台。

Electron

SQLite

Inspector

统一管理平台：治理流程，不替代 Agent

DevFlow 不自研所有 Agent 能力，而是给外部 Agent 套上权限、证据和团队视图。

遇到的问题

OpenCode / opencode 等外部 Agent 能做执行，但团队仍需要权限控制、证据捕获和协作状态。

解决思路

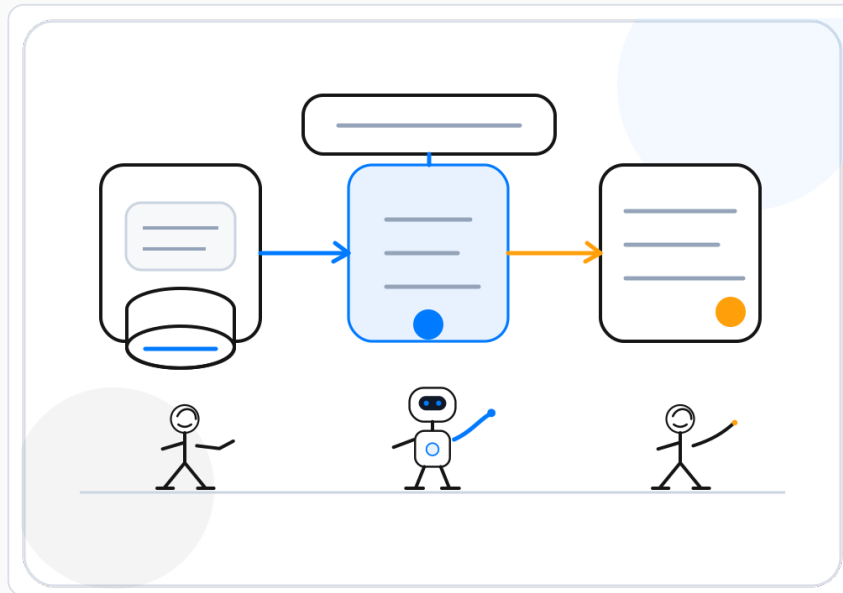
通过 Coding Agent Adapter 对接外部能力；DevFlow 负责上下文、权限、Trace、Evidence、团队摘要。

落地动作

Electron + SQLite 处理本地敏感信息；API + Postgres 支撑团队管理；Adapter 连接外部执行引擎。

阶段结果

平台能统一管理项目、成员、风险、成本和 PR 交付包，同时不绑定单一执行引擎。



漫画示意：本地、Adapter、团队平台之间边界清楚。

本页结论

外部 Agent 可以替换，团队治理能力不能丢。

Adapter

Postgres

Team View

这不是终点，而是一条可验证的路径

作为 Demo，它还不完整；但已经验证了一条从知识到交付治理的方向。

已经完成

知识底座、多仓连接、US 拆解、设计原型、React / Electron 工作台和管理平台框架。

核心变化

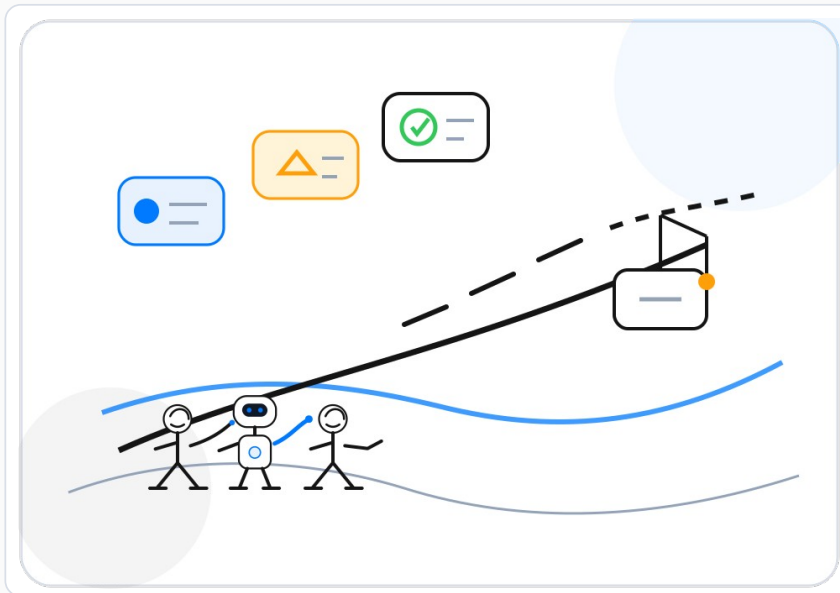
从知识碎片到团队知识底座；从黑盒调用到证据链；从个人 Agent 使用到团队治理。

下一步

接入真实 OpenCode / opencode Agent，完善知识审核流，强化 Gate 与团队运行机制。

真实定位

当前仍是探索型 Demo，还有很多不完善；它的价值是提供一种有益探索。



漫画示意：桥还在建设，但方向已经被验证。

本页结论

抛砖引玉：给团队一条把 AI 研发产品化、流程化、可复盘的路径。

探索型 Demo

可验证

下一步